47 — DEF Reduction Deep Dive (where "-X% DEF" actually lands)

> **Question (LordFaramire, 10/09/2026):** "Can we see in the game code where the defense reduction goes?" Screenshot context: battle stat panel showing **Final DEF**, a flat **DEF Total** bucket (Initial / Equip / Adventure Advancement / Enchant / Avian / Treasure) and a **Base DEF Total %** bucket (Pal / Skill / …), alongside a pal **Deploy Effect: "Reduce all enemies' DEF by 24%"** and a support reply stating there is **no stacking cap** on DEF reduction in PvP or PvE.
>
> **Config data:** all 908 tables decoded from `uploads/bundle-firstload-res.zip` via `decode_config_data.py` (733,618 records → `data/tables/`, gitignored). `Buff.json` = 2,476 rows.
>
> **Sources (all byte offsets into `game_script.js`, the 18 MB minified bundle in this repo, 2026-02-28 capture):** `MetaAttrib.ts` @13543031 · `BuffAttrib.ts` @8511078 · `BuffAttribContinue.ts` @8515461 · `BuffAttribCondition.ts` @8512748 · `BuffAttribConvert.ts` @8517832 · `SkillRunner.addBuff` @17059152 · `BuffCtr.ts` @8540148 · `BuffClear.ts` @8535706 · `HurtUtil.normalHurt` @12441031 · `setPlayerAttrib` @8294499 · `ConfigBuff.ts` @9441185

TL;DR

1. **The panel buckets are not where the debuff lands.** "DEF Total" and "Base DEF Total %" are out-of-battle assembly, resolved server-side. The battle unit is handed one number per attribute.
2. In battle, a "-X% DEF" debuff goes into the `_time` **multiplier of the `def` (1024) MetaAttrib** — it scales the already-assembled **Final DEF**, after every panel bucket.
3. **Multiplicative DEF modifiers are additive with each other**, not compounding: `_time = 1 + Σ(modifiers)`. Two -24% debuffs give ×0.52, not ×0.5776.
4. **There is no lower clamp anywhere.** `MetaAttrib` caps the top (`up_limit`) and nothing caps the bottom. DEF can reach 0 and go **negative**.
5. Negative DEF is not a no-op — in `normalHurt` the term is `max(roundInt(ATK − DEF×(1+def_coe)), 1)`, so a negative DEF **adds** to the attacker's effective ATK.
6. "No stacking cap" is true **across different sources** and false within one: of the 11 DEF reductions in this build, only one is `mutex 0` (free stacking). The flagship PvP one — Star Hero **King** (30013), 27%→54% by star — is `mutex 1`, i.e. a second copy replaces the first. See §4b.

1. What the battle unit actually receives

`setPlayerAttrib` (@8294499) builds every combat attribute (`module == 1`) from the server payload:

```
t.setPlayerAttrib = function(t, i, a) {
    for (var f of configAttribute.getDataByList("module", 1)) {
        var c = configAttribute.getDataByKey(f.id),
            v = new MetaAttrib(c);
        // PvE-only stats zeroed out in PvP chapters, everything else straight from attr_list
        (0 != s.pve || f.id != pve_dam && f.id != pve_resist)
            ? v.baseValue = this.getAttrib(f.id, t.attr_list)
            : v.baseValue = 0;
        i.attribs[c.id] = v
    }
}
t.getAttrib = function(t, i) {            // simple k/v lookup, no assembly
    for (var s of i) if (s.k == t) return roundInt(Number(s.v));
    return 0
}
```

There is **no client-side code that combines the flat bucket with the % bucket** — `ConfigAttr_source` (@9398753) carries only `id / name / order`, i.e. it is a label table for the panel rows. The buckets exist purely for display; `attr_list[1024]` is already **Final DEF**, and that is what `baseValue` becomes.

Sanity check against the screenshot: `57,006 × ~318,150 ≈ 18.14 B` vs the displayed Final DEF of **18,121.8M** — consistent with `Final DEF = flat DEF Total × (1 + Base DEF Total %)` to display precision. (The % figure is partly covered by the markup circle, so this is an inference from the numbers, not from code.)

---

2. MetaAttrib — the only place an attribute value is produced

`MetaAttrib.ts` @13543031:

```
function MetaAttrib(config, copy) {
    this._baseValue = 0; this._addValue = 0; this._addExtraValue = 0; this._time = 1; ...
}
_calculateValue = function(base) {
    var v = roundInt(roundInt(base + this._addValue) * this._time + this._addExtraValue);
    if (0 != this.config.up_limit) v = Math.min(v, this.config.up_limit);   // ← cap is TOP ONLY
    if (2 == this.config.num_type) v = round(v / 1e4);
    return v ?? 0
}
addValue      = function(x) { this._addValue      = round(this._addValue + x);      this._change = !0 }
addMultiples  = function(x) { this._time          = round(this._time + x);          this._change = !0 }
addExtraValue = function(x) { this._addExtraValue = round(this._addExtraValue + x); this._change = !0 }
multiple      = function(x) { this._time          = this._time * x;                 this._change = !0 }
```

So the full in-battle DEF expression is:

```
DEF = min( roundInt( roundInt(FinalDEF + Σflat) × (1 + Σpercent) + Σpost ), up_limit )
```

- `up_limit` for `def` (1024) is **0 = uncapped** (`Attribute.json`: `{"id":1024,"key":"def","num_type":1,"up_limit":0,"add_type":0}`).
- **No `Math.max(0, …)` exists in this path.** Negative results pass straight through.

Which attributes actually are capped

`MetaAttrib` reads `ConfigAttribute.up_limit`. Eleven attributes have a non-zero value — none of them DEF:

ID	key	up_limit	=
1014 / 1015	att_hpsteal / skill_hpsteal	10000	100%
1018 / 1019 / 1020 / 1021	att_resist / skill_resist / partner_resist / resist	8000	80%
1034 / 1035	double_hit_def / counter_def	8000	80%
1042	control_res	10000	100%
1052	boss_def	8000	80%
1059	season_cannon_att_def	6000	60%

Note this is a **different mechanism** from the `battle_up_limit: [[1008, 8000]]` global in `data/constants/attribute_caps.json`. That one never reaches `MetaAttrib` — it is applied in `checkHit` (@12446261) as a cap on the computed evasion probability, and only outside PvE:

```
var u = round(Global.battle_up_limit[0][1] / 1e4);        // 0.8
...
1 != configChapter_type.getDataByKey(c).pve && (m = Math.min(s, u));   // PvP only
```

So `miss` (1008) is capped as a rolled probability, not as a stat, and `03_ATTRIBUTES.md`'s "only capped attribute" line understates the real `up_limit` list above.

---

3. How a "-24% DEF" deploy effect is encoded

`BuffAttrib.ts` @8511078 — the `action: "attrib"` buff class:

```
alloc = function(cfg) {
    buff.config = cfg;
    buff._id          = cfg.param1;      // attribute id − 1024 for DEF
    buff._isMultiples = 2 == cfg.param2; // 2 = multiplicative, else flat
    buff._value       = cfg.param3;      // base magnitude, typically -1
    return buff
}
onBegin = function() {
    var meta = this.owner.data.getAttribMeta(this._id);
    this._lastValue = round(this._value *
        this.runner.cast.data.getSkillFactAttrValue(this.skillPar, this.runner.useSkill.config.id, 1044));
    this._isMultiples ? meta.addMultiples(this._lastValue) : meta.addValue(this._lastValue)
}
onDestroy = function() {  // exact reversal when the buff ends
    this._lastValue && (this._isMultiples ? meta.addMultiples(-this._lastValue) : meta.addValue(-
this._lastValue))
}
```

So a 24% pal deploy debuff is `param1=1024, param2=2, param3=-1, skillPar=0.24` → `addMultiples(-0.24)` → `_time: 1 → 0.76`. This is the same shape as the snail slow documented in 43_SNAIL_PAL_SPEED_REDUCTION_DEEP_DIVE.md (`param1=1009`).

The `getSkillFactAttrValue(skillPar, skillId, 1044)` wrapper

@17730276. The magnitude is not raw `skillPar` — it is scaled first:

```
getSkillFactAttrValue = function(par, skillId, attrGroup) {
    var n = round(par + round(skillAttr(skillId, attrGroup) + skillAttr(0, attrGroup)));  // additive
    for (var cfg of configAttribute.getDataByList("group", attrGroup))                    // then
multiplicative
        n = round(n * round(1 + round(roundInt(skillAttr(skillId, cfg.id) + skillAttr(0, cfg.id)) /
1e4)));
    return n
}
```

This is the hook the Beastmaster Lv50 passive ("Enhance Pal Deploy Effects by **20%**", skill 2106 — see 32_CLASS_SKILLS_REFERENCE.md) rides on: a 24% deploy debuff becomes **28.8%** for a Beastmaster, because the enhancement scales `skillPar` before it ever reaches `addMultiples`.

Other channels into the same attribute

Buff class	Offset	Channel	Notes
`BuffAttrib`	8511078	`addValue` / `addMultiples`	the standard flat or % modifier
`BuffAttribContinue`	8515461	same	re-applies `value × skillPar` every `param4` ms for `param5[0]` s — a ramp; accumulates in `_totaladdvalue`, **no cap**, fully reversed on expiry
`BuffAttribCondition`	8512748	same	magnitude scales off HP%/speed delta; **this one does clamp**, via `param5` `_limit` min/max
`BuffAttribConvert` / `BuffPetConvert`	8517832 / 8576604	`addExtraValue`	converts one stat into another; lands after the multiplier

Nothing else in the bundle writes to a `MetaAttrib`. All of these pool into the same three accumulators, which is why order of application is irrelevant.

---

4. Stacking — what the engine actually enforces

Stacking is governed per-buff by `mutex` (field 7) and `add_max` (field 8) in `ConfigBuff` (@9441185), enforced in `SkillRunner.addBuff` (@17059152):

`mutex`	Behaviour
0 (or unset)	stacks freely — every application is a separate live instance
1	`stopBuffById(id)` first → **replace** any existing instance of that same buff id
2	skipped entirely if the target already has that buff id
3	stacks up to `add_max`; every application refreshes all durations; at the cap the oldest running instance is stopped
4	one per caster — a second application from the same caster is skipped
5	one per caster — a second application from the same caster only refreshes duration

Crucially, every one of these checks is keyed on `config.id` (`getBuff(e)`, `getBuffListById(e)`). Two different sources of DEF reduction are two different buff ids, so no mutex rule can ever suppress one against the other — they coexist and both land in `_time`. That is the engine-level confirmation of the support reply: **no global cap on DEF-reduction stacking exists**, and none could, because the sum happens inside `MetaAttrib._time` where nothing is clamped. Same-source stacking is the only thing that can be limited, and only by that buff's own `mutex` / `add_max`.

What does and does not stop an attrib debuff

```
var C = ["dizz", "ban_skil", "throw_hit", "bound", "ban_act"];      // SkillRunner.ts @17051640
...
if ((t.statectr.notControlled > 0 || t.statectr.invincible > 0) && C.includes(s.action)) return null;
```

- **CC immunity does not block attribute debuffs.** `"attrib"` is not in `C`. Likewise `CONTROL_RES` (1042) only shortens `"dizz"` / `"ban_act"` durations — it does not touch `"attrib"`. (This contradicts §5 of doc 43, which states the snail slow is blocked by *`notControlled`* / *`invincible`* because it is group 3. The gate in the shipped code is on *`action`*, not *`group`*. Logged in 98_DISCREPANCIES.)
- **`IGNORE_BUFFIDS`** (BuffGroupType 330) blocks a debuff only if its `param5` list names that buff id.
- **Cleanse** (`BuffClear` @8535706) has two modes: `param1 == 0` → `buffCtr.removeBuff(n)`, which matches on `buffType` (= `config.group`, set in `Buff.config` setter @8501296) and so can wipe a whole group; `param1 == 1` → `stopBuffById(n)`, which matches a specific buff id. Either way the removal reverses the modifier exactly (`onDestroy` subtracts `_lastValue`).
- `battle_attribute_def` (1079) and `battle_resist_def` (1080) look like they should resist this. They do not — both appear **only** as ConfigUnit field getters and in the `AttribDefine` enum. No battle code reads them.

Pierce is a different system

`calArmorAndBlock` (@12441031, in the `normalHurt` tail) modifies the **resistance** term (`att_resist` / `partner_resist` / ...), capped at ±50% per hit, and never touches `def`. Armor Penetration is not DEF reduction — see 08_PIERCE_BLOCK_INSPIRE_SUPPRESS.md.

4b. Every DEF modifier that exists in this build (verified against Buff.json)

`Buff.json`, 2,476 rows. Buffs whose `action` is in the attrib family (`attrib` 891, `attrib_continue` 17, `attrib_condition` 21, `attrib_convert` 9) and whose `param1 == 1024`:

Mode	Count
`attrib`, `param2=2` (multiplicative), `param3 > 0` — DEF **buffs**	41
`attrib`, `param2=2` (multiplicative), `param3 < 0` — DEF **reductions**	**11**
`attrib`, `param2=1` (flat), positive	1
`attrib_condition`, positive	1

All 11 reductions share `param2=2, param3=-1`, i.e. magnitude comes entirely from `skillPar`:

buff	mutex	add_max	group	source	self or enemy
50489	3	7	504	skills 51373-75 "Divine Champion" (artifact-tier skill, 3 levels)	self (ATK-for-DEF trade)
50666	3	6	4	**Mount 407**, skin skill 5061 "Thunder Rush"	**self** — "Afterwards, DEF is reduced by ##7% while ATK increases by ##8%, stacking up to 6 times"
51182	3	5	4	**Artifact 21**, skin level 10 skill 51182 "Duck Swirl Strike"	self
51572	3	15	4	not referenced by any skill effect in this build	—
90002	2	0	401	skills 10001-10014 "Family GvG Win Streak Buff"	enemy (mode penalty)
100002	2	0	401	skills 13001-13105 / 15001-15005 / 31001-31005 "Parking Defense / Plunder / Capture Minions Debuff" — "ATK and DEF -##1%" (15/35/55/75/95%)	enemy (mode penalty)
190008	2	0	401	skills 20003-20012 "Seasonal Win Streak Debuff" — 15% → 90% by streak	self (win-streak handicap)
240156	1	0	4	skill 24039 "Lovestruck Prank"	enemy
249009	1	0	4	effect 249082 (no skill reference in this build)	—
270268	1	0	4	**Angel 30013 "King"**, skill 27041	**enemy**
4155020	**0**	0	4	skill 4155238 (event)	enemy

The one real PvP enemy-DEF debuff: Star Hero "King" (30013)

```
Skill 27041 → effect 270411 → buffGroup entry [1, 270268, 4, 5]
                                ^buffId        ^par ^duration(s)
desc 700023165: "Every 15s after the battle starts, all enemies lose ##1% ATK and ##2% DEF
                 for 5s (cannot be cleansed)."
```

Star	ATK −	DEF −
1	13.5%	**27%**
5	19.5%	39%
10	27%	**54%**

Buff 270268 is `mutex 1` (replace). Two instances of this buff never coexist — a second application calls `stopBuffById` on the first. So the "no stacking cap" answer is right across different sources and wrong if read as "two Kings stack". Of the 11, only **4155020** is `mutex 0` (unlimited free stacking); four are `mutex 3` (hard-capped at `add_max` 5/6/7/15, all self-inflicted); three are `mutex 1` (replace); three are `mutex 2` (one instance, ever).

skillPar is XOR-encoded — and decodes exactly

`Skill_level.skillCoefficient` is protected with the standard config key **24455**, and decodes to the value in /10000. Verified against the display params:

```
King L1 : raw [23233, 21771] ^ 24455 = [1350, 2700] → 13.5%, 27%   (desc_parm [13.5, 27]) ✓
King L10: raw [21771, 19103] ^ 24455 = [2700, 5400] → 27%,   54%   (desc_parm [27, 54])   ✓
```

That decoded value is the `skillPar` handed to `addBuff`, so the chain `skillCoefficient ^ 24455 / 10000 → getSkillFactAttrValue → × param3(-1) → addMultiples` is closed end to end.

About the 24% in the screenshot

No string matching "Reduce all enemies' DEF by 24%" exists in this build's `Language_en.json`. The closest verified effect is King's 27%→54%. The screenshot is dated 2026-10-09, roughly seven months after the 2026-02-28 config capture, so the effect is almost certainly content added since. Everything above about where the number lands still applies; only the specific source and its `mutex` are unconfirmed for it.

A note on effect chains

Some descriptions bundle effects from several skills. The Guardian line ("reduce all enemies' ATK, DEF, ATK SPD, Energy Regen and Pal ATK SPD by ##3%", desc 405621, skill 300602) resolves through `addbuff_to_target` → buff 300013, which targets **1082** `total_dam_def`, not 1024 — i.e. the "Final DMG RES" half of the text. Tracing a DEF debuff always means following the buff chain to a `param1 == 1024` leaf, not trusting the English blurb.

---

5. What happens at 0 DEF — and below

`HurtUtil.normalHurt` (@12441031):

```
var f = roundInt( Math.max(roundInt(o - u * (1 + g)), 1) * round(A * round(1 - m)) );
//                         ^ATK  ^DEF    ^def_coe
```

The `max(…, 1)` floors the **whole subtraction**, not DEF itself. Walk the cases with a 24% debuff:

Sources applied	`_time`	DEF (as fraction of Final DEF)	`ATK − DEF×(1+def_coe)`
0	1.00	100%	ATK − DEF
1	0.76	76%	ATK − 0.76·DEF
2	0.52	52%	ATK − 0.52·DEF
4	0.04	4%	ATK − 0.04·DEF
5	**−0.20**	**−20%**	**ATK + 0.20·DEF·(1+def_coe)**

So "reduce them to 0 DEF" is not the ceiling of this effect — it is the halfway point. Past ~4.17 stacked instances of −24%, the defender's DEF stat becomes a **damage bonus for the attacker**, scaled by the attacker's own `def_coe`… which is the defender's stat, so the more DEF-amplification the target has stacked, the worse the flip hurts them.

Same pool, opposite sign: in-battle **DEF buffs** (e.g. "increases DEF by 32%") are also `addMultiples` on 1024, so they net out additively against debuffs — `1 + 0.32 − 0.24 = 1.08`. A defender does not multiply their way out of a stacked debuff; they can only out-add it.

---

6. Practical notes / caveats

- The `max(…, 1)` floor means the ATK−DEF term is a **cliff**, not a curve. If `ATK < DEF×(1+def_coe)`, every hit lands on the floor of 1 before multipliers, and the first DEF debuff that pushes the target below your ATK is worth vastly more than the previous ones. Against a 4.72 B Final DEF wall, DEF reduction is doing nothing at all until your ATK is in range — then it is doing everything.
- **Verified since first draft:** buff ids, `mutex` and `add_max` for every DEF modifier in the build — see §4b. Decode with `python3 decode_config_data.py <capture> --output data/tables`; the capture lives at `uploads/bundle-firstload-res.zip` (the 12 MB `config/datas` blob is in there, not in `lom.joynetgame.com.zip`, which is assets only).
- **Still open:** whatever source produces the 24% figure in the screenshot postdates this capture, so its `mutex` is unknown. If it is `mutex 1` like King's, a second copy replaces rather than stacks.
- Whether a line reads "DEF" or "Final DEF" (e.g. the Angel support skill "increases Final DEF by 8%") does not correspond to separate attributes — DEF is only 1024. "Final DMG RES" is a genuinely different attribute, 1082 `total_dam_def`.